

# PROYECTO INTEGRADOR [35%]

## Asignatura: Calidad de Software 2026A

### Sistema de Gestión de Pruebas Integrales para Aplicación Web

Aplicación de Estándares, Métricas y Herramientas Modernas de QA/QC

#### Instrucciones Generales del Proyecto

Documento de Enunciado y Especificaciones

## Tabla de Contenidos

- Descripción General del Proyecto
- Objetivos del Proyecto
- Temas Cubiertos
- Requisitos Técnicos
- Especificaciones del Proyecto
- Estructura del Repositorio
- Criterios de Evaluación
- Entregables
- Consideraciones Adicionales

## 1. Descripción General del Proyecto

El proyecto final de la asignatura **Calidad de Software** consiste en desarrollar un Sistema Integral de Gestión de Pruebas para una aplicación web en Java. Este proyecto integra todos los conceptos, estándares y herramientas estudiadas en el curso, demostrando competencia en el aseguramiento y control de calidad (QA/QC) de software moderno.

El proyecto requiere crear una aplicación web Java con funcionalidades específicas, acompañada de un sistema completo de pruebas automatizadas, análisis de cobertura, pruebas de rendimiento y pruebas de interfaz de usuario, todo orquestado mediante Docker y controlado con Git.

## Alcance del Proyecto

- Desarrollo de una aplicación web Java funcional

- Suite completa de pruebas unitarias (JUnit)
- Pruebas funcionales de interfaz (Selenium WebDriver)
- Pruebas de rendimiento y carga (JMeter)
- Análisis de cobertura de código (SonarQube)
- Orquestación e integración mediante Docker y GitHub

## 2. Objetivos del Proyecto

### Objetivo General

Aplicar de forma integral los conceptos de calidad de software, estándares internacionales (ISO/IEC 25010, ISO 9001, CMMI, ISO/IEC 29119) y herramientas modernas de QA/QC en el desarrollo y validación de una aplicación web funcional en Java.

### Objetivos Específicos

- Implementar pruebas unitarias utilizando JUnit con una cobertura mínima del 80% en lógica de negocio
- Desarrollar pruebas funcionales de interfaz de usuario con Selenium WebDriver utilizando el patrón Page Object Model
- Ejecutar pruebas de rendimiento y carga con JMeter, identificando puntos críticos
- Analizar la calidad del código con SonarQube, reportando métricas de defecto y cobertura
- Documentar un plan de pruebas integral según ISO/IEC 29119
- Implementar pipelines de CI/CD usando Docker y GitHub para automatizar la ejecución de pruebas

## 3. Temas Cubiertos

### Unidad 0: Introducción a Calidad de Software

- Conceptos fundamentales de calidad de software
- Evolución histórica y grandes referentes en la industria
- Aplicación en el contexto del proyecto

### Unidad 1: Estándares y Métricas de Calidad

- Modelo de Calidad ISO/IEC 25010 y características de calidad
- QA vs QC: Diferencias y aplicación práctica
- Estándares: ISO 9001, CMMI (Capability Maturity Model Integration)
- Normativa ISO/IEC 29119 para procesos de prueba
- Métricas de calidad: defect density, cobertura de código, MTTR (Mean Time To Recovery)
- Interpretación y reporte de métricas en SonarQube

### Unidad 2: Fundamentos y Técnicas de Testing

- Niveles de prueba: unitarias, integración, sistema, aceptación
- Tipos de pruebas: funcionales, no funcionales, seguridad
- Técnicas de diseño: caja negra y caja blanca
- Gestión de pruebas: planificación, documentación, trazabilidad
- Herramientas y automatización

## Unidad 3: Herramientas Prácticas y Automatización

- JUnit: Pruebas unitarias en Java (Assertions, Fixtures, Suites)
- Selenium WebDriver: Automatización de pruebas funcionales
- Page Object Model (POM): Patrón para tests de Selenium escalables
- JMeter: Pruebas de rendimiento, carga y estrés
- SonarQube: Análisis estático, métricas y calidad de código
- Docker: Contenedores para ambiente reproducible
- GitHub: Control de versión y CI/CD

## 4. Requisitos Técnicos

### Lenguaje y Marco de Trabajo

- Lenguaje: Java (JDK 11 o superior)
- Framework web: Spring Boot 2.x (recomendado) o similar
- Base de datos: H2, PostgreSQL o MySQL

### Herramientas de Prueba

- JUnit 4 o 5: Pruebas unitarias
- Selenium WebDriver 4.x: Pruebas de interfaz
- JMeter 5.x: Pruebas de rendimiento
- SonarQube (versión comunitaria): Análisis de calidad
- Maven o Gradle: Gestión de dependencias y construcción

### Infraestructura

- Docker: Contenedores para aplicación, base de datos y herramientas
- Docker Compose: Orquestación de múltiples contenedores
- GitHub: Repositorio de código y CI/CD (GitHub Actions)
- Git: Control de versión local

### Requerimientos del Sistema

- SO: Linux
- RAM: Mínimo 8 GB recomendado para Docker + SonarQube
- Espacio en disco: Mínimo 10 GB

## 5. Especificaciones del Proyecto

### 5.1 Aplicación Web

**La aplicación web debe ser funcional y contener al menos las siguientes características:**

- Autenticación y autorización de usuarios
- CRUD (Create, Read, Update, Delete) de al menos una entidad principal
- Validación de entrada de datos
- Interfaz de usuario HTML/CSS/JavaScript funcional
- Gestión de excepciones adecuada
- API REST (opcional pero recomendado) para la integración con pruebas

### 5.2 Pruebas Unitarias (JUnit)

- Cobertura mínima del 80% en clases de lógica de negocio
- Pruebas para casos exitosos, casos límite y casos de error
- Uso de fixtures (@Before, @After o @BeforeEach, @AfterEach)
- Aserciones significativas y descriptivas
- Suites de pruebas organizadas y documentadas
- Reporte de cobertura generado con JaCoCo o similar

### 5.3 Pruebas Funcionales (Selenium WebDriver)

- Mínimo 5 casos de prueba funcionales que cubre flujos críticos
- Implementación del patrón Page Object Model (POM)
- Pruebas en navegadores Chrome y Firefox
- Manejo de esperas explícitas (WebDriverWait)
- Reportes de ejecución de pruebas
- Documentación de escenarios de prueba

### 5.4 Pruebas de Rendimiento (JMeter)

- Plan de prueba con múltiples escenarios: normal (1-5 usuarios), carga (10-50 usuarios) y estrés
- Medición de tiempos de respuesta, throughput y tasa de error
- Análisis de resultados en gráficos y reportes
- Identificación de cuellos de botella y puntos críticos
- Recomendaciones de optimización basadas en resultados

### 5.5 Análisis de Calidad (SonarQube)

- Integración de SonarQube en Docker
- Análisis de código con reporte de errores, vulnerabilidades y code smells
- Métricas clave: defect density, cobertura de líneas, duplicación de código
- Rating de seguridad (A, B, C, D, E)
- Documentación de hallazgos y plan de mejora

### 5.6 Documentación y Evidencia

- Plan de Pruebas (según ISO/IEC 29119): alcance, estrategia, recursos
- Especificación de Pruebas: casos de prueba detallados con precondiciones y postcondiciones
- Reporte de Ejecución: resultados, defectos encontrados, trazabilidad
- Reporte de Métricas: cobertura, defect density, MTTR estimado
- README.md en el repositorio con instrucciones de instalación y ejecución

## 6. Estructura del Repositorio

El repositorio GitHub debe estar organizado de la siguiente manera:

- /app: Código fuente de la aplicación Java
- /app/src/main: Código de producción
- /app/src/test: Pruebas unitarias (JUnit)
- /tests/selenium: Pruebas funcionales (Selenium WebDriver + POM)
- /tests/jmeter: Planes de prueba de rendimiento (.jmx)
- /docker: Archivos Dockerfile y docker-compose.yml

- /reports: Reportes generados (cobertura, JMeter, SonarQube)
- /docs: Documentación (planes, especificaciones, evidencia)
- /.github/workflows: Configuración de CI/CD (GitHub Actions)
- pom.xml o build.gradle: Configuración de dependencias
- README.md: Instrucciones generales del proyecto

## 7. Criterios de Evaluación

### Rúbrica de Evaluación

| Criterio                  | Excelente (5)           | Bueno (4)                    | Regular (3)                |
|---------------------------|-------------------------|------------------------------|----------------------------|
| Estructura y Organización | Perfecta                | Bien organizado              | Parcialmente organizado    |
| Cobertura JUnit           | $\geq 85\%$             | 75-84%                       | 60-74%                     |
| Pruebas Selenium          | $\geq 5$ casos          | 4 casos                      | 3 casos                    |
| JMeter + Reportes         | Análisis completo       | Análisis adecuado            | Análisis básico            |
| SonarQube + Métricas      | Integrado y documentado | Integrado parcialmente       | Solo reporte               |
| Docker y CI/CD            | Completamente funcional | Funcional con fallos menores | Funcional con limitaciones |
| Documentación             | Completa y clara        | Completa pero con defectos   | Incompleta                 |

### Notas Importantes

- Calidad de código: Se evaluará la claridad, mantenibilidad y adherencia a buenas prácticas
- Documentación: Debe ser completa, clara y demostrar comprensión de conceptos
- Reproducibilidad: El proyecto debe ejecutarse sin errores en diferentes ambientes
- Automatización: Todas las pruebas deben ejecutarse automáticamente mediante Docker y GitHub Actions

## 8. Entregables

### Entrega 1: Código y Estructura Base (30% de la nota)

- Repositorio GitHub con estructura completa
- Aplicación web funcional en Spring Boot
- Docker Compose configurado
- README.md con instrucciones

### Entrega 2: Pruebas y Automatización (30% de la nota)

- Suite de pruebas unitarias (JUnit) con cobertura  $\geq 80\%$
- Pruebas funcionales con Selenium WebDriver + POM
- Planes de prueba de rendimiento con JMeter
- Integración de SonarQube en Docker
- GitHub Actions para CI/CD

### Entrega 3: Documentación y Reportes (20% de la nota)

- Plan de Pruebas (ISO/IEC 29119)
- Especificación de Pruebas
- Reporte de Ejecución
- Reportes de Métricas (cobertura, defect density, análisis SonarQube)
- Conclusiones y lecciones aprendidas
- Realice un trabajo escrito de mínimo de 20 páginas que incluya: portada, tabla de contenido, índice de tablas y figuras, introducción, objetivos, desarrollo, resultados obtenidos, conclusiones y referencias. El documento debe estar en formato PDF y se debe utilizar APA 7.

### Entrega 4: Sustentación del Proyecto Final (20% de la nota)

- Realice la sustentación del proyecto final con su grupo de trabajo. La presentación debe incluir 10 diapositivas y la duración es de 15 minutos.

## 9. Consideraciones Adicionales

### Mapa de Alineación con Temas del Curso

| Unidad | Tema                   | Aplicación en Proyecto                     |
|--------|------------------------|--------------------------------------------|
| 0      | Conceptos de Calidad   | Fundamentación de QA/QC en desarrollo      |
| 1      | Estándares y Métricas  | ISO/IEC 25010, SonarQube, defect density   |
| 2      | Fundamentos de Testing | Niveles y tipos de pruebas implementados   |
| 3      | Herramientas Prácticas | JUnit, Selenium, JMeter, SonarQube, Docker |

### Recomendaciones para el Éxito

- Inicie con la estructura base y Docker lo antes posible
- Implemente pruebas unitarias mientras desarrolla la aplicación
- Use datos de prueba realistas en Selenium y JMeter
- Documente su trabajo a medida que avanza
- Realice commits frecuentes en GitHub con mensajes descriptivos
- Pruebe la reproducibilidad del proyecto en diferente máquina antes de entregar
- Consulte la documentación oficial de cada herramienta cuando sea necesario

### Recursos Complementarios

- JUnit Official Documentation: <https://junit.org/>
- Selenium WebDriver Documentation: <https://www.selenium.dev/documentation/>
- JMeter User Manual: <https://jmeter.apache.org/usermanual/>
- SonarQube Documentation: <https://docs.sonarqube.org/>
- Docker Documentation: <https://docs.docker.com/>

- GitHub Actions: <https://docs.github.com/actions>
- ISO/IEC 25010: Software product quality model (disponible en sitios académicos)

## **Fechas Importantes**

Las fechas de entrega serán comunicadas por el docente según el cronograma de la asignatura.